

Operating Systems MCQ Question Bank

Interview & Exam Preparation

200 Multiple-Choice Questions with Answers & Explanations

Covering every topic across the full Operating Systems syllabus — process/thread management, CPU scheduling, synchronization, deadlocks, memory management, virtual memory, and file systems/disk scheduling.

PART 1	OS Fundamentals, Processes & Threads	40 Qs
PART 2	CPU Scheduling	40 Qs
PART 3	Process Synchronization	40 Qs
PART 4	Deadlocks & Memory Management	40 Qs
PART 5	Virtual Memory, File Systems & Disk Scheduling	40 Qs

How to use this question bank: Each question has four options (A–D). The correct option is marked with a checkmark (✓) and shown in green, followed by a one-line explanation. Cover the answer line with your hand, attempt each question, then check yourself against the explanation. Questions are grouped by topic to match a full Operating Systems course, so you can revise part-by-part or attempt the whole bank as a timed mock test.

Topics covered: OS types & functions (batch, time-sharing, distributed, real-time), system calls, kernel vs user mode, monolithic vs microkernel, process states & PCB, context switching, threads & multithreading models, IPC, CPU scheduling (FCFS, SJF, Priority, Round Robin, Multilevel Queues), process synchronization (semaphores, mutexes, monitors, Producer-Consumer, Readers-Writers, Dining Philosophers), deadlocks (conditions, prevention, avoidance, Banker's Algorithm, detection & recovery), memory management (fragmentation, paging, segmentation), virtual memory (demand paging, page replacement, thrashing, working set), and file systems & disk scheduling (allocation methods, directory structures, FCFS/SSTF/SCAN/C-SCAN/LOOK).

Q1. An Operating System primarily acts as:

- A) A programming language
- ✓ B) An intermediary between users/applications and computer hardware
- C) A type of database
- D) A network cable

Answer: B — The OS manages hardware resources and provides a convenient environment for programs.

Q2. Which is NOT a main function of an OS?

- A) Process management
- B) Memory management
- ✓ C) Compiling source code into an executable
- D) File management

Answer: C — Compiling is a compiler's job, not a core OS function.

Q3. In a Batch Operating System:

- A) Users interact with jobs in real time
- ✓ B) Jobs are collected and run in batches with no user interaction during execution
- C) Only one job can ever be stored
- D) Threads are the primary unit of scheduling

Answer: B — Batch OS processes queued jobs without live user interaction.

Q4. A Time-Sharing (multitasking) OS is designed primarily to:

- A) Run one job to completion before starting another
- ✓ B) Give multiple interactive users the illusion of simultaneous access via rapid CPU switching
- C) Eliminate the need for a scheduler
- D) Only run on a single CPU with one process

Answer: B — Time-sharing slices CPU time across users/tasks for responsiveness.

Q5. A Real-Time OS (RTOS) is defined by its ability to:

- A) Run batch jobs only
- ✓ B) Guarantee task completion within a strict, predictable deadline
- C) Ignore all deadlines
- D) Run only on mainframes

Answer: B — RTOS guarantees timely responses, critical for systems like airbags or industrial control.

Q6. A Distributed OS manages:

- A) A single isolated machine only
- ✓ B) Multiple physically separate computers so they appear as one coherent system
- C) Only virtual machines
- D) Only mobile devices

Answer: B — Distributed OS presents multiple machines as a unified system to users.

Q7. The key difference between multiprogramming and multitasking is:

- A) They are identical concepts
- ✓ B) Multiprogramming maximizes CPU utilization; multitasking additionally focuses on fast interactive response via time-slicing
- C) Multitasking only works on single-core CPUs
- D) Multiprogramming requires no memory

Answer: B — Multiprogramming just avoids CPU idling; multitasking specifically targets responsive time-sharing.

Q8. A program becomes a process when:

- A) It is written in a high-level language
- ✓ B) It is loaded into memory and is actively executing
- C) It is compiled but not run
- D) It is deleted from disk

Answer: B — A process is a program in execution, with its own PC, registers, and resources.

Q9. System calls are needed because:

- A) User programs can directly access all hardware safely
- ✓ B) User-mode programs cannot directly touch hardware or protected OS memory
- C) They slow down the OS unnecessarily
- D) They are optional in modern OSes

Answer: B — System calls provide the controlled interface for user programs to request OS services.

Q10. Which is a category of system calls?

- ✓ A) Process control
- B) Web browsing
- C) Video editing
- D) Spreadsheet formatting

Answer: A — Process control (create/terminate/wait) is a standard system call category.

Q11. In User mode, a running program:

- A) Has unrestricted hardware access
- ✓ B) Has restricted access and cannot execute privileged instructions directly
- C) Can modify the kernel freely
- D) Bypasses all OS protection

Answer: B — User mode restricts direct hardware/privileged instruction access.

Q12. Kernel mode (supervisor mode) allows:

- A) Only read access to memory
- ✓ B) Unrestricted access to hardware and execution of any CPU instruction
- C) No access to hardware at all
- D) Access limited to a single user

Answer: B — Kernel mode grants the OS full control over hardware and instructions.

Q13. A Monolithic kernel is characterized by:

- ✓ A) Running almost all OS services in a single large program in kernel space
- B) Running every service as an isolated user-space process
- C) Having no device drivers at all
- D) Being slower than a microkernel due to message passing

Answer: A — Monolithic kernels bundle most services together in kernel space for speed.

Q14. A Microkernel is characterized by:

- A) Running all services in kernel space for speed
- ✓ B) Keeping minimal services in kernel space, with most others as user-space processes communicating via messages
- C) Never using message passing
- D) Being identical to a monolithic kernel

Answer: B — Microkernels minimize kernel-space code, favoring modularity and robustness.

Q15. The five typical process states include all of the following EXCEPT:

- A) New
- B) Ready
- ✓ C) Compressed
- D) Terminated

Answer: C — The standard states are New, Ready, Running, Waiting, Terminated — not 'Compressed'.

Q16. A process moves from Ready to Running when:

- ✓ A) It is dispatched by the scheduler onto the CPU
- B) It requests I/O
- C) It terminates
- D) It is created

Answer: A — The dispatcher moves a Ready process onto the CPU (Running state).

Q17. A process moves from Running to Waiting when:

- A) It finishes execution
- ✓ B) It needs to wait for an I/O operation or event
- C) It is dispatched
- D) It is admitted into the system

Answer: B — Blocking on I/O or an event moves a process to the Waiting state.

Q18. A Process Control Block (PCB) stores:

- A) Only the process's source code
- ✓ B) Process ID, state, program counter, registers, and scheduling/memory info
- C) The entire contents of the hard disk
- D) Only network configuration

Answer: B — The PCB holds all information needed to manage and resume a process.

Q19. A context switch involves:

- A) Deleting the old process permanently
- ✓ B) Saving the current process's state and loading another process's saved state
- C) Formatting the hard disk
- D) Installing a new OS

Answer: B — Context switching saves/restores process state to switch which process runs.

Q20. Context switching is considered overhead because:

- A) It always improves performance
- ✓ B) The CPU does no useful user work during the switch itself
- C) It never happens in modern OSes
- D) It only occurs once per day

Answer: B — Time spent switching context is time not spent running actual process work.

Q21. A thread is best described as:

- A) A completely isolated process with its own memory space
- ✓ B) The smallest unit of CPU execution within a process, sharing that process's resources
- C) A type of file system
- D) A hardware component

Answer: B — Threads share a process's memory/resources but have their own PC, registers, and stack.

Q22. Compared to processes, threads within the same process:

- A) Cannot communicate with each other at all
- ✓ B) Share code, data, and open files, making creation/switching cheaper
- C) Each require a completely separate address space
- D) Are always slower to create

Answer: B — Shared memory space makes thread operations cheaper than process operations.

Q23. User-level threads are managed by:

- A) The kernel directly
- ✓ B) A user-space thread library, without kernel involvement
- C) Hardware interrupts only
- D) The BIOS

Answer: B — User-level threads are handled entirely in user space by a thread library.

Q24. A drawback of user-level threads is that:

- A) They are always faster than kernel threads with no downside
- ✓ B) A blocking system call in one thread can block the entire process
- C) They require kernel modification
- D) They cannot be created dynamically

Answer: B — Since the kernel only sees one thread of control, one blocking call can stall all threads.

Q25. Kernel-level threads are scheduled by:

- A) A user-space library only
- ✓ B) The OS kernel directly
- C) The compiler
- D) The BIOS

Answer: B — Kernel-level threads are individually recognized and scheduled by the kernel.

Q26. In the Many-to-One threading model:

- A) Each user thread maps to its own kernel thread
- ✓ B) Many user threads map to a single kernel thread
- C) There are no user threads at all
- D) Kernel threads outnumber user threads

Answer: B — Many-to-One multiplexes many user threads onto one kernel thread.

Q27. In the One-to-One threading model:

- ✓ A) Each user thread maps to its own kernel thread, enabling true parallelism
- B) All threads share one kernel thread
- C) Threads cannot run in parallel
- D) Kernel threads are unnecessary

Answer: A — One-to-One gives true parallelism since each user thread has a kernel thread.

Q28. The Many-to-Many threading model:

- ✓ A) Maps many user threads onto a smaller or equal number of kernel threads
- B) Is identical to Many-to-One
- C) Cannot be implemented in practice
- D) Requires zero kernel threads

Answer: A — Many-to-Many balances flexibility and parallelism between the two extremes.

Q29. Inter-Process Communication (IPC) is needed because:

- A) Processes automatically share all memory by default
- ✓ B) Processes are normally isolated and need explicit mechanisms to exchange data
- C) IPC is only used within a single thread
- D) The OS forbids any process interaction

Answer: B — Processes don't share memory by default, so IPC mechanisms enable communication.

Q30. Shared memory IPC is generally faster than message passing because:

- A) It always uses the kernel for every access
- ✓ B) Once set up, processes access shared memory directly without kernel mediation
- C) It requires no synchronization
- D) It works only across networks

Answer: B — Direct access without kernel involvement makes shared memory IPC very fast.

Q31. Message passing IPC is often preferred over shared memory when:

- A) Processes are on the same machine only
- ✓ B) Processes are distributed across a network, since it works naturally there
- C) Speed is the only concern
- D) Synchronization is never needed

Answer: B — Message passing generalizes naturally to distributed/networked systems.

Q32. Which best distinguishes a process from a thread?

- A) A thread is an isolated unit of resources; a process shares resources with siblings
- ✓ B) A process is an isolated unit of resources; a thread is an independent execution unit sharing resources with sibling threads
- C) They are exactly the same concept
- D) Threads always require more memory than processes

Answer: B — This is the standard textbook distinction between processes and threads.

Q33. Which OS type would best suit an airbag deployment controller?

- A) Batch OS
- ✓ B) Real-Time OS
- C) Distributed OS
- D) Time-sharing OS

Answer: B — Airbags need guaranteed, deadline-bound responses — the definition of an RTOS use case.

Q34. The 'New' process state refers to:

- A) A process that has just terminated
- ✓ B) A process that is being created
- C) A process currently running on the CPU
- D) A process waiting for I/O

Answer: B — New = the process is in the process of being created.

Q35. Which best explains why two privilege modes (user/kernel) exist?

- A) To make programs run faster
- ✓ B) To protect the system from buggy or malicious user programs corrupting critical resources
- C) To reduce the number of system calls needed
- D) To eliminate the need for a scheduler

Answer: B — Privilege separation is fundamentally a protection mechanism.

Q36. A Hybrid kernel (e.g., Windows NT) attempts to:

- A) Be purely monolithic with no user-space services
- ✓ B) Combine benefits of monolithic and microkernel designs
- C) Avoid using system calls entirely
- D) Function without any kernel mode

Answer: B — Hybrid kernels blend monolithic speed with microkernel modularity.

Q37. Multiprocessor operating systems primarily manage:

- A) A single CPU system only
- ✓ B) Systems with multiple CPUs working together
- C) Only virtual machines
- D) Only mobile devices

Answer: B — Multiprocessor OSes coordinate work across multiple physical CPUs.

Q38. A 'Network OS' is specifically designed to:

- A) Run entirely without any network connectivity
- ✓ B) Manage and provide services to interconnected computers over a network
- C) Replace the need for any OS at all
- D) Only run batch jobs

Answer: B — Network OSes manage networked machine coordination and resource sharing.

Q39. Which best describes the relationship between a process's threads and its open files?

- A) Each thread has a completely separate set of open files
- ✓ B) All threads within the same process share the same open files
- C) Threads cannot access files at all
- D) Only the first thread can access files

Answer: B — Shared resources (files, memory, code) are a defining feature of threads in one process.

Q40. Why is switching between threads of the same process cheaper than switching between processes?

- A) Threads require reloading the entire memory map
- ✓ B) Threads share the same address space, so less context needs to be saved/loaded
- C) Thread switches always involve disk I/O
- D) There is no difference in cost

Answer: B — Shared address space reduces the state that must be saved/restored on a thread switch.

Q41. The main goal of CPU scheduling is to:

- A) Minimize memory usage only
- ✓ B) Decide which ready process gets the CPU next, maximizing utilization and performance
- C) Format the hard disk
- D) Eliminate the need for processes

Answer: B — CPU scheduling picks the next process to run to optimize system performance.

Q42. Which is NOT a standard CPU scheduling criterion?

- A) CPU utilization
- ✓ C) Screen resolution
- B) Turnaround time
- D) Waiting time

Answer: C — Screen resolution is unrelated to CPU scheduling performance metrics.

Q43. Turnaround time is defined as:

- A) Time spent only in the ready queue
- ✓ B) Total time from submission to completion of a process
- C) Time to the first response only
- D) Time spent in the Waiting state only

Answer: B — Turnaround time spans the entire submission-to-completion duration.

Q44. Waiting time refers to:

- A) Time spent executing on the CPU
- ✓ B) Total time a process spends in the ready queue
- C) Time spent in I/O only
- D) Time until the process is created

Answer: B — Waiting time is time spent ready but not running.

Q45. Response time is especially important for:

- A) Batch systems only
- ✓ B) Interactive/time-sharing systems
- C) Systems with a single process
- D) Systems with no scheduler

Answer: B — Fast response time matters most where users interact live with the system.

Q46. In non-preemptive scheduling:

- A) The CPU can be forcibly taken from a running process at any time
- ✓ B) Once given the CPU, a process keeps it until it voluntarily releases it
- C) Processes never use the CPU
- D) Time quantum is always used

Answer: B — Non-preemptive scheduling waits for voluntary release (completion or I/O block).

Q47. In preemptive scheduling:

- ✓ A) A running process can be forcibly interrupted and moved back to ready
- B) Processes can never be interrupted
- C) There is no scheduler needed
- D) Only one process ever runs

Answer: A — Preemptive scheduling allows forced context switches (e.g., time quantum expiry).

Q48. FCFS (First-Come, First-Served) scheduling is:

- A) Preemptive by nature
- ✓ B) Non-preemptive, servicing processes strictly in arrival order
- C) Based on shortest burst time
- D) Based on priority levels

Answer: B — FCFS is simple and non-preemptive, following arrival order exactly.

Q49. The 'convoy effect' in FCFS refers to:

- A) All processes finishing at the same time
- ✓ B) A long process at the front causing many shorter processes behind it to wait a long time
- C) Processes running in parallel efficiently
- D) Zero waiting time for all processes

Answer: B — A long job blocking short jobs behind it is the classic convoy effect.

Q50. SJF (Shortest Job First) scheduling selects:

- A) The process that arrived first
- ✓ B) The ready process with the smallest estimated CPU burst time
- C) The process with the highest priority number
- D) A random ready process

Answer: B — SJF picks the process needing the least CPU time next.

Q51. SJF is considered optimal because it:

- A) Always has zero waiting time
- ✓ B) Minimizes average waiting time among all scheduling algorithms
- C) Requires no burst time estimate
- D) Never causes starvation

Answer: B — SJF is provably optimal for minimizing average wait time, though it needs accurate estimates and can starve long jobs.

Q52. Preemptive SJF is also known as:

- A) Round Robin
- ✓ B) Shortest Remaining Time First (SRTF)
- C) FCFS
- D) Priority Scheduling

Answer: B — Preemptive SJF = SRTF, switching to a shorter newly-arrived job immediately.

Q53. A risk of SJF scheduling is:

- ✓ A) Starvation of long processes if short jobs keep arriving
- B) Guaranteed fairness for all processes
- C) Impossibility of implementation
- D) Zero context switching

Answer: A — Long jobs can be perpetually delayed if short jobs keep arriving ahead of them.

Q54. Priority scheduling can lead to which problem for low-priority processes?

- A) Faster execution
- ✓ B) Starvation — never getting the CPU
- C) Guaranteed immediate execution
- D) Automatic termination

Answer: B — Continuous arrival of higher-priority processes can starve low-priority ones.

Q55. Aging is a technique used to solve:

- A) Deadlock
- ✓ B) Starvation, by gradually increasing a waiting process's priority
- C) Thrashing
- D) External fragmentation

Answer: B — Aging raises priority over time so waiting processes eventually get scheduled.

Q56. Round Robin scheduling assigns each process:

- A) Unlimited CPU time
- ✓ B) A small, fixed time quantum before being preempted
- C) Priority based only on arrival time
- D) No CPU time at all

Answer: B — RR gives every process a fixed time slice before moving to the next.

Q57. If the time quantum in Round Robin is too small:

- A) The algorithm behaves like FCFS
- ✓ B) Overhead from frequent context switching dominates
- C) There is no effect on performance
- D) Processes never get CPU time

Answer: B — Too small a quantum causes excessive context-switch overhead.

Q58. If the time quantum in Round Robin is too large:

- ✓ A) Round Robin behavior degrades toward FCFS
- B) Context switch overhead becomes infinite
- C) The scheduler crashes
- D) All processes finish instantly

Answer: A — A very large quantum makes RR act like FCFS since processes rarely get preempted.

Q59. Multilevel Queue scheduling:

- A) Uses a single queue for all processes
- ✓ B) Splits the ready queue into separate queues by process type, each with fixed priority
- C) Prevents any process from ever running
- D) Requires no scheduling algorithm

Answer: B — Multilevel Queue assigns processes permanently to different queues by category.

Q60. Multilevel Feedback Queue scheduling improves on Multilevel Queue by:

- A) Never letting processes change queues
- ✓ B) Allowing processes to move between queues based on their behavior (e.g., CPU usage)
- C) Using only one queue
- D) Ignoring process priority entirely

Answer: B — MLFQ dynamically adjusts a process's queue based on observed behavior.

Q61. A CPU-bound process that keeps using its full time quantum in MLFQ is typically:

- A) Promoted to a higher-priority queue immediately
- ✓ B) Demoted to a lower-priority queue
- C) Terminated automatically
- D) Given unlimited priority

Answer: B — Heavy CPU users get pushed to lower-priority queues in MLFQ.

Q62. Given processes with burst times 5, 3, 8, 4 arriving in that order, FCFS scheduling would run them in the order:

- A) Sorted by shortest burst first
- ✓ B) Exactly in arrival order: 5, 3, 8, 4
- C) Sorted by longest burst first
- D) In reverse arrival order

Answer: B — FCFS always follows arrival order, not burst time.

Q63. Given the same processes (burst times 5, 3, 8, 4, all arriving at time 0), non-preemptive SJF would run them in the order:

- A) 5, 3, 8, 4
- ✓ B) 3, 4, 5, 8
- C) 8, 5, 4, 3
- D) 4, 8, 3, 5

Answer: B — SJF sorts by shortest burst first: 3, 4, 5, 8.

Q64. A scheduling algorithm is 'starvation-free' if:

- A) No process ever has to wait
- ✓ B) Every ready process is guaranteed to eventually get the CPU
- C) Only one process can ever run
- D) The algorithm never uses preemption

Answer: B — Starvation-free means every process eventually gets service, however long.

Q65. Which algorithm is most likely to cause starvation without an aging mechanism?

- A) Round Robin
- B) FCFS
- ✓ C) Priority Scheduling
- D) None of these can starve

Answer: C — Plain priority scheduling can indefinitely starve low-priority processes.

Q66. CPU utilization as a scheduling criterion aims to:

- A) Keep the CPU idle as much as possible
- ✓ B) Keep the CPU as busy as possible
- C) Maximize the number of idle CPUs
- D) Reduce the number of ready processes to zero

Answer: B — High CPU utilization means the CPU is rarely idle.

Q67. Throughput in scheduling refers to:

- ✓ A) The number of processes completed per unit time
- B) The physical speed of the CPU clock
- C) The size of the ready queue
- D) The number of I/O devices

Answer: A — Throughput measures process completion rate.

Q68. Which scheduling algorithm is most appropriate for a general-purpose interactive time-sharing system?

- A) FCFS
- ✓ B) Round Robin
- C) Pure SJF (non-preemptive)
- D) None of these are used in practice

Answer: B — Round Robin's fairness and bounded wait fit interactive time-sharing well.

Q69. The 'dispatcher' in scheduling is responsible for:

- A) Deciding the scheduling policy
- ✓ B) Actually giving control of the CPU to the process selected by the scheduler
- C) Managing memory allocation only
- D) Writing to the log file

Answer: B — The dispatcher performs the mechanical work of switching the CPU to the chosen process.

Q70. Dispatch latency refers to:

- A) The time to compile a program
- ✓ B) The time it takes the dispatcher to stop one process and start another
- C) The time a user waits for a webpage
- D) The time to format a disk

Answer: B — Dispatch latency is the overhead of the context switch performed by the dispatcher.

Q71. In priority scheduling, a lower priority NUMBER conventionally means:

- A) Lower actual priority (runs last)
- ✓ B) Higher actual priority (often runs first)
- C) No priority at all
- D) The process is terminated

Answer: B — By common convention, smaller priority numbers often indicate higher priority.

Q72. Which best describes a 'CPU burst'?

- A) A period during which a process performs I/O only
- ✓ B) A period during which a process uses the CPU before needing I/O or completing
- C) The total lifetime of a process
- D) A type of scheduling algorithm

Answer: B — A CPU burst is a contiguous stretch of CPU usage between I/O waits.

Q73. Which factor makes SJF difficult to implement perfectly in practice?

- ✓ A) CPU burst times are hard to know in advance
- B) All burst times are always known exactly
- C) SJF requires no scheduler
- D) SJF only works with one process

Answer: A — SJF needs (often estimated) future burst times, which aren't perfectly predictable.

Q74. A key trade-off in choosing Round Robin's time quantum is:

- ✓ A) Between responsiveness (short quantum) and context-switch overhead (avoided by longer quantum)
- B) Between memory size and disk size
- C) Between file systems and databases
- D) There is no trade-off

Answer: A — Quantum size balances responsiveness against context-switching costs.

Q75. Which of these criteria would a batch system prioritize LEAST?

- A) Throughput
- B) CPU utilization
- ✓ C) Response time
- D) Turnaround time

Answer: C — Batch systems have no live user, so fast interactive response time matters least.

Q76. A scheduling algorithm that always picks the process closest to a chosen ideal (e.g., shortest job) but risks unfairness is characteristic of:

- A) FCFS
- ✓ B) SJF
- C) Round Robin
- D) FIFO memory allocation

Answer: B — SJF optimizes average wait time but can be unfair to longer jobs.

Q77. Multilevel Queue scheduling is best suited when:

- A) All processes are identical in nature
- ✓ B) Processes can be clearly categorized (e.g., system, interactive, batch) with different needs
- C) There is only ever one process running
- D) No priority differences exist among processes

Answer: B — Multilevel Queue fits naturally when distinct process categories exist.

Q78. Compared to Multilevel Queue, Multilevel Feedback Queue is generally considered:

- A) Less flexible since processes are fixed to one queue
- ✓ B) More flexible and adaptive, since it can adjust a process's queue based on behavior
- C) Impossible to implement
- D) Identical in every way

Answer: B — MLFQ's dynamic queue movement gives it more adaptability.

Q79. Given equal-length processes with no arrival delay, which algorithm behaves essentially like FCFS?

- ✓ A) Round Robin with a very large time quantum
- B) SJF with tiny bursts
- C) Priority scheduling with unique priorities
- D) None of the above

Answer: A — A large-enough time quantum means Round Robin rarely preempts, mimicking FCFS.

Q80. Why might average waiting time be a poor sole indicator of a good scheduling algorithm for interactive systems?

- ✓ A) It ignores response time, which matters more to interactive users
- B) It's always identical to turnaround time
- C) It cannot be measured
- D) It only applies to single-processor systems

Answer: A — Interactive systems care deeply about response time, not just average wait.

Q81. A race condition occurs when:

- A) Processes never access shared data
- ✓ B) The outcome depends unpredictably on the timing of concurrent access to shared data
- C) Only one process exists in the system
- D) All processes run strictly sequentially

Answer: B — Race conditions arise from timing-dependent concurrent access to shared data.

Q82. The Critical Section is best defined as:

- A) The entire program of a process
- ✓ B) The part of a process's code that accesses shared resources
- C) Code that never runs
- D) The operating system kernel itself

Answer: B — The critical section is specifically the shared-resource-accessing code segment.

Q83. Mutual Exclusion, as a requirement for critical section solutions, means:

- A) Multiple processes can be in their critical sections simultaneously
- ✓ B) Only one process can be in its critical section at a time
- C) No process can ever enter its critical section
- D) Processes must alternate turns strictly

Answer: B — Mutual exclusion ensures exclusive access to the critical section.

Q84. The Progress requirement for critical section solutions ensures that:

- A) A process can be denied entry forever with no reason
- ✓ B) The decision of who enters next cannot be postponed indefinitely among waiting processes
- C) Only the OS decides who enters, arbitrarily
- D) No process ever wants to enter

Answer: B — Progress guarantees a timely decision among those actually waiting.

Q85. The Bounded Waiting requirement ensures that:

- A) A process may wait forever with no limit
- ✓ B) There's a limit on how many times others can enter before a waiting process's turn comes
- C) Only two processes can ever compete for the resource
- D) Bounded waiting is optional, not required

Answer: B — Bounded waiting caps how many times other processes can cut in line.

Q86. A semaphore is best described as:

- A) A GUI element
- ✓ B) An integer variable accessed only via atomic wait() and signal() operations
- C) A file storage format
- D) A type of index

Answer: B — Semaphores are synchronization variables manipulated via P/V (wait/signal).

Q87. The wait() (P) operation on a semaphore:

- A) Always increments the semaphore
- ✓ B) Decrements the semaphore, blocking the process if it becomes negative
- C) Deletes the semaphore
- D) Has no effect on the calling process

Answer: B — *wait()/P()* decrements the value and may block the caller.

Q88. The signal() (V) operation on a semaphore:

- A) Decrements the semaphore only
- ✓ B) Increments the semaphore, potentially waking a waiting process
- C) Terminates the process
- D) Deletes all waiting processes

Answer: B — *signal()/V()* increments the value and can unblock a waiting process.

Q89. A binary semaphore can only take the values:

- A) Any integer
- ✓ B) 0 or 1
- C) Negative numbers only
- D) Only positive even numbers

Answer: B — *Binary semaphores are restricted to 0 and 1, acting like a simple lock.*

Q90. A counting semaphore is used to:

- A) Enforce strict binary locking only
- ✓ B) Manage a pool of multiple identical resource instances
- C) Prevent all concurrency
- D) Store user passwords

Answer: B — *Counting semaphores track availability across multiple resource units.*

Q91. The key difference between a mutex and a semaphore is:

- A) They are identical in every respect
- ✓ B) A mutex enforces ownership (only the locker can unlock); a semaphore is a more general signaling counter
- C) A semaphore can only be binary
- D) A mutex can be signaled by any process, unlike a semaphore

Answer: B — *Mutex = strict ownership lock; semaphore = general-purpose counting signal.*

Q92. A monitor is best described as:

- A) A physical display screen
- ✓ B) An abstract data type bundling shared data with procedures, ensuring automatic mutual exclusion
- C) A type of semaphore with no automation
- D) A hardware interrupt handler

Answer: B — *Monitors encapsulate data + procedures with built-in mutual exclusion.*

Q93. Condition variables inside a monitor are used with which operations?

- A) read() and write()
- ✓ B) wait() and signal()
- C) lock() and unlock() only
- D) commit() and rollback()

Answer: B — *Monitors use condition variables with wait()/signal() for complex synchronization.*

Q94. Compared to semaphores, monitors are generally considered:

- A) More error-prone since compilers can't help
- ✓ B) Easier and safer to use correctly, since mutual exclusion is automatic
- C) Identical in safety and use
- D) Obsolete and never used

Answer: B — Monitors reduce the risk of forgetting to release/signal correctly.

Q95. In the Producer-Consumer problem, the 'empty' semaphore tracks:

- A) The number of filled buffer slots
- ✓ B) The number of free/empty buffer slots
- C) The number of consumers waiting
- D) The producer's priority level

Answer: B — 'empty' counts available (unfilled) slots in the bounded buffer.

Q96. In the Producer-Consumer problem, the 'full' semaphore tracks:

- A) The number of empty slots
- ✓ B) The number of filled buffer slots ready for consumption
- C) The buffer's total capacity only
- D) The number of producers

Answer: B — 'full' counts slots currently holding unconsumed items.

Q97. In the Producer-Consumer problem, the mutex semaphore is used to:

- A) Track empty slots
- ✓ B) Protect the actual insert/remove operation on the shared buffer
- C) Count producers only
- D) Replace the need for 'empty' and 'full'

Answer: B — The mutex ensures only one process manipulates the buffer's internal state at a time.

Q98. The Readers-Writers problem allows:

- A) Only one reader at a time, ever
- ✓ B) Multiple readers to read simultaneously, but writers need exclusive access
- C) Writers and readers to always work simultaneously with no restriction
- D) No concurrent access of any kind

Answer: B — Readers can share access; writers require exclusivity from all others.

Q99. In the Readers-Writers problem, why can multiple readers proceed simultaneously?

- A) Reading modifies the data, requiring exclusion
- ✓ B) Reading doesn't change the data, so concurrent reads cause no conflict
- C) Readers always have higher priority than writers
- D) Only one reader is ever allowed by definition

Answer: B — Reads are non-destructive, so concurrent reads are safe.

Q100. The Dining Philosophers problem is primarily used to illustrate:

- A) Network latency issues
- ✓ B) Deadlock and starvation risks in resource-sharing systems
- C) File allocation strategies
- D) CPU scheduling fairness only

Answer: B — It's a classic illustration of deadlock/starvation from resource contention.

Q101. In the Dining Philosophers problem, deadlock can occur if:

- A) Every philosopher picks up their right fork only
- ✓ B) Every philosopher simultaneously picks up their left fork first, then waits forever for the right
- C) No philosopher ever tries to eat
- D) Forks are unlimited in number

Answer: B — Everyone grabbing left forks first creates a circular wait -> deadlock.

Q102. A common fix for the Dining Philosophers deadlock is to:

- A) Give every philosopher two sets of forks
- ✓ B) Have philosophers pick up forks in a fixed global order, breaking circular wait
- C) Remove all forks from the table
- D) Let philosophers eat without forks

Answer: B — A global fork-pickup ordering prevents the circular wait condition.

Q103. Which of these is a valid solution requirement for the Critical Section Problem?

- A) Starvation
- ✓ B) Mutual Exclusion
- C) Race Condition
- D) Deadlock

Answer: B — Mutual Exclusion is one of the three formal CSP requirements (with Progress, Bounded Waiting).

Q104. A binary semaphore used purely for locking is functionally similar to a:

- A) Counting semaphore with unlimited value
- ✓ B) Mutex
- C) File descriptor
- D) Page table

Answer: B — A binary semaphore behaves like a simple mutex lock.

Q105. If a producer tries to insert into a completely full buffer without proper synchronization, the result could be:

- A) Perfectly safe automatic queuing
- ✓ B) Data being overwritten or corrupted (buffer overflow logic error)
- C) The buffer automatically resizing
- D) Nothing; this can never happen

Answer: B — Without proper semaphore checks, inserting into a full buffer causes errors.

Q106. Two threads incrementing a shared counter without synchronization can result in:

- A) Always the mathematically correct final result
- ✓ B) A lost update due to a race condition
- C) An immediate crash every time
- D) The counter resetting to zero automatically

Answer: B — Concurrent unsynchronized increments are the classic lost-update race condition example.

Q107. Which statement about Progress and starvation is TRUE?

- A) Satisfying Progress alone guarantees no starvation for any individual process
- ✓ B) Progress ensures a decision is made among waiting processes, but doesn't alone guarantee bounded waiting
- C) Progress and Bounded Waiting are the same requirement
- D) Progress means every process runs simultaneously

Answer: B — Progress avoids indefinite postponement of the overall decision, but starvation of a specific process needs bounded waiting too.

Q108. A monitor's automatic mutual exclusion means:

- A) Programmers must manually insert lock/unlock calls everywhere
- ✓ B) Only one process can be active inside the monitor at any time, enforced by the language/runtime
- C) Any number of processes can execute monitor procedures simultaneously
- D) Monitors have no synchronization at all

Answer: B — The monitor construct itself enforces exclusivity, unlike raw semaphores.

Q109. Why is a race condition considered dangerous even if it happens 'rarely'?

- A) It never actually affects program correctness
- ✓ B) It can cause silent, hard-to-reproduce data corruption depending on unlucky timing
- C) It always crashes the program immediately and visibly
- D) It only affects single-threaded programs

Answer: B — Race conditions are notoriously hard to detect/reproduce since they depend on timing.

Q110. Which best explains why a mutex should only be unlocked by the thread that locked it?

- A) To make debugging harder
- ✓ B) To enforce clear ownership and prevent accidental/unauthorized unlocking by unrelated threads
- C) Because mutexes don't support unlocking at all
- D) It is not actually a requirement anywhere

Answer: B — Ownership semantics prevent bugs from unrelated threads improperly releasing a lock.

Q111. A semaphore initialized to 3 for a resource pool means:

- A) Only one process can ever use the resource
- ✓ B) Up to 3 units of the resource can be used concurrently before wait() blocks further requests
- C) The resource is unlimited
- D) The resource is completely unavailable

Answer: B — A counting semaphore starting at 3 permits 3 concurrent acquisitions before blocking.

Q112. Which scenario is an example that specifically needs the Readers-Writers solution?

- A) A single-user text editor with no sharing
- ✓ B) A shared configuration file read by many services but occasionally updated by an admin
- C) A completely read-only static file with no writers ever
- D) A file accessed by exactly one process only

Answer: B — Many concurrent readers plus occasional exclusive writer access is the Readers-Writers scenario.

Q113. What happens if a process forgets to call signal() after wait() on a mutex semaphore?

- A) Nothing; the system self-corrects
- ✓ B) The resource remains permanently locked, potentially causing indefinite blocking of other processes
- C) The semaphore resets automatically after 1 second
- D) Other processes ignore the lock and proceed anyway

Answer: B — A missing signal() is a classic bug causing permanent lock-out (deadlock-like symptom).

Q114. Validation (optimistic) concurrency approaches assume:

- A) Conflicts are extremely common, so locks should always be held
- ✓ B) Conflicts are rare, so checks can be deferred to just before commit
- C) No synchronization is ever necessary
- D) Only one process can ever run

Answer: B — Optimistic approaches (also used in DB transactions) assume low conflict likelihood.

Q115. Which of the following is the best real-world analogy for a monitor?

- A) A single shared physical key handed to everyone at once
- ✓ B) A storeroom whose door automatically locks itself and only allows one person in at a time
- C) An unlocked room with no restrictions
- D) A room that is permanently locked with no entry ever

Answer: B — The automatic, built-in exclusivity is the defining feature of a monitor.

Q116. Which best describes 'atomic' in the context of wait()/signal() operations?

- A) They can be interrupted partway through by another process
- ✓ B) They execute as a single, indivisible operation with no interleaving allowed
- C) They always take exactly one second
- D) They only work on binary semaphores

Answer: B — Atomicity ensures wait()/signal() cannot be partially executed or interleaved unsafely.

Q117. The main challenge the Critical Section Problem tries to solve is:

- A) How to format a hard disk safely
- ✓ B) How to let concurrent processes safely share data without corrupting it
- C) How to compile code faster
- D) How to choose the best CPU scheduling algorithm

Answer: B — The Critical Section Problem is fundamentally about safe concurrent data sharing.

Q118. A deadlock arising specifically from synchronization primitives (like in Dining Philosophers) is prevented by breaking which condition?

- A) Mutual Exclusion always
- ✓ B) Typically Circular Wait, via ordering resource acquisition
- C) CPU utilization
- D) Response time

Answer: B — Ordering fork/resource acquisition breaks the circular wait condition.

Q119. Which is TRUE about semaphore signal() calls?

- A) They can only be called by the same process that called wait()
- ✓ B) Any process can call signal(), even one that never called wait()
- C) They must always be paired with a mutex lock
- D) They can never wake up another process

Answer: B — Unlike a mutex, a semaphore's signal() isn't restricted to the original 'locker'.

Q120. Which best describes why Dining Philosophers uses an odd/circular resource layout?

- A) To make the problem trivial to solve
- ✓ B) To model circular contention for shared resources, illustrating deadlock risk realistically
- C) Because forks must always be arranged in a circle in real kitchens
- D) It has no bearing on the deadlock concept

Answer: B — The circular arrangement directly models circular wait, the deadlock condition it illustrates.

Q121. A deadlock occurs when:

- A) A process finishes execution normally
- ✓ B) A group of processes are each waiting for a resource held by another in the group, with no progress possible
- C) Only one process exists in the system
- D) A process voluntarily releases all resources

Answer: B — Deadlock: circular resource waiting with no possible progress.

Q122. Which is NOT one of the four necessary conditions for deadlock?

- A) Mutual Exclusion
- ✓ C) Bounded Waiting
- B) Hold and Wait
- D) Circular Wait

Answer: C — The four conditions are Mutual Exclusion, Hold and Wait, No Preemption, Circular Wait.

Q123. The 'Hold and Wait' condition means:

- A) A process releases all resources before requesting more
- ✓ B) A process holds at least one resource while waiting to acquire additional resources
- C) Resources can be preempted freely
- D) No process ever holds any resource

Answer: B — Hold and Wait: holding one resource while waiting for another.

Q124. The 'No Preemption' condition means:

- A) Resources can be forcibly taken from a process at any time
- ✓ B) A resource can only be released voluntarily by the process holding it
- C) All resources are always available
- D) Preemption is required for deadlock

Answer: B — No Preemption: resources aren't forcibly taken away.

Q125. The 'Circular Wait' condition means:

- A) Processes request resources in increasing order only
- ✓ B) A closed chain of processes exists where each waits for a resource held by the next
- C) No two processes ever wait for each other
- D) Only one process can request resources

Answer: B — Circular Wait: a cyclic chain of waiting processes.

Q126. For deadlock to occur, how many of the four necessary conditions must hold simultaneously?

- A) Just one
- ✓ C) All four
- B) Any two
- D) None are required

Answer: C — All four conditions must hold together for deadlock to be possible.

Q127. A Resource Allocation Graph edge from a resource to a process indicates:

- A) The process is requesting the resource
- ✓ B) The resource is currently allocated to the process
- C) The process has terminated
- D) No relationship exists

Answer: B — Resource → Process edge = 'allocated'.

Q128. A Resource Allocation Graph edge from a process to a resource indicates:

- A) The resource is allocated to the process
- ✓ B) The process is requesting the resource
- C) The resource has been deleted
- D) The process is idle

Answer: B — Process → Resource edge = 'requested'.

Q129. If each resource type has only ONE instance, a cycle in the Resource Allocation Graph means:

- A) Deadlock is impossible
- ✓ B) Deadlock definitely exists
- C) The system is perfectly safe
- D) More resources are needed only

Answer: B — With single-instance resource types, a cycle guarantees deadlock.

Q130. Deadlock prevention works by:

- A) Detecting deadlock after it happens and killing a process
- ✓ B) Ensuring at least one of the four necessary conditions can never hold
- C) Ignoring deadlocks entirely
- D) Increasing the number of CPUs only

Answer: B — Prevention structurally eliminates one of the four necessary conditions.

Q131. A common deadlock prevention strategy targeting Circular Wait is to:

- A) Allow any resource request order
- ✓ B) Impose a strict global ordering on resource types and require requests in increasing order
- C) Never allow more than one process to run
- D) Remove all resources from the system

Answer: B — Ordered resource requests structurally prevent circular wait.

Q132. Deadlock avoidance differs from prevention in that it:

- A) Eliminates one of the four conditions outright
- ✓ B) Carefully evaluates each request to keep the system in a safe state, without eliminating any condition
- C) Ignores deadlocks completely
- D) Only works with a single resource type

Answer: B — Avoidance dynamically ensures safety rather than removing a necessary condition.

Q133. The Banker's Algorithm is used for:

- A) CPU scheduling
- ✓ B) Deadlock avoidance by checking if granting a request keeps the system in a safe state
- C) File allocation
- D) Page replacement

Answer: B — Banker's Algorithm simulates resource allocation to ensure safety before granting requests.

Q134. A 'safe state' in the Banker's Algorithm means:

- A) No process can ever finish
- ✓ B) There exists at least one order in which all processes can complete without deadlock
- C) All resources are currently free
- D) The system has crashed

Answer: B — A safe state guarantees a completion order exists for all processes.

Q135. Deadlock detection and recovery differs from prevention/avoidance in that it:

- A) Never allows deadlock to occur
- ✓ B) Lets the system allocate resources freely, then periodically checks for and resolves deadlocks
- C) Requires no algorithm at all
- D) Only works on single-user systems

Answer: B — Detection/recovery is reactive rather than proactive.

Q136. A common deadlock recovery technique is:

- A) Ignoring the deadlock forever
- ✓ B) Terminating one or more processes in the deadlock cycle
- C) Adding more deadlocked processes
- D) Rebooting the entire network

Answer: B — Process termination (or resource preemption) is a standard recovery method.

Q137. The Wait-Die deadlock avoidance scheme is:

- A) Preemptive — it forcibly rolls back other transactions
- ✓ B) Non-preemptive — a younger transaction requesting an older one's resource is killed
- C) Identical to Wound-Wait
- D) Not based on timestamps at all

Answer: B — Wait-Die is non-preemptive and timestamp-based, killing younger requesters.

Q138. The Wound-Wait deadlock avoidance scheme is:

- A) Non-preemptive only
- ✓ B) Preemptive — an older transaction can force a rollback of a younger one
- C) Identical to Wait-Die
- D) Unrelated to transaction timestamps

Answer: B — Wound-Wait preemptively 'wounds' (rolls back) younger transactions.

Q139. Internal fragmentation occurs when:

- A) Free memory is scattered in small non-contiguous chunks
- ✓ B) A process is allocated more memory than it actually needs, wasting the excess inside the block
- C) All memory is perfectly utilized
- D) No memory is ever allocated

Answer: B — Internal fragmentation is wasted space INSIDE an allocated block.

Q140. External fragmentation occurs when:

- A) A single block has wasted space inside it
- ✓ B) Total free memory is sufficient but scattered in small chunks too small individually to satisfy a request
- C) Memory is never freed
- D) Only one process is ever loaded

Answer: B — External fragmentation is scattered free space between allocated blocks.

Q141. A fix for external fragmentation is:

- A) Increasing internal fragmentation intentionally
- ✓ B) Compaction, or switching to paging
- C) Removing all indexes
- D) Disabling the scheduler

Answer: B — Compaction consolidates free space; paging eliminates external fragmentation by design.

Q142. In paging, physical memory is divided into:

- A) Segments of varying size
- ✓ B) Fixed-size blocks called frames
- C) Single large contiguous blocks per process
- D) Sectors used only for boot data

Answer: B — Paging divides physical memory into equal-size frames.

Q143. In paging, a process's logical address space is divided into:

- A) Variable-size segments
- ✓ B) Fixed-size blocks called pages
- C) A single block only
- D) Directories only

Answer: B — Paging divides logical memory into equal-size pages, matching frame size.

Q144. The page table's role is to:

- A) Store user passwords
- ✓ B) Map logical page numbers to physical frame numbers
- C) Replace the need for a CPU
- D) Store SQL queries

Answer: B — The page table performs the logical-to-physical address translation.

Q145. Paging eliminates external fragmentation because:

- ✓ A) All pages are the same fixed size, so any free frame fits any page
- B) Paging doesn't use frames at all
- C) Paging always leaves memory unallocated
- D) Pages are never actually loaded into memory

Answer: A — Uniform page/frame size means any free frame can hold any page — no gaps.

Q146. Paging can still suffer from a small amount of:

- A) External fragmentation
- ✓ B) Internal fragmentation, in a process's last, partially-filled page
- C) No fragmentation at all
- D) Deadlock

Answer: B — The last page of a process may not be fully used, causing minor internal fragmentation.

Q147. A Translation Lookaside Buffer (TLB) is used to:

- A) Replace the page table entirely
- ✓ B) Cache recent page-table lookups for faster address translation
- C) Store the entire disk contents
- D) Manage CPU scheduling

Answer: B — The TLB is a small, fast cache speeding up virtual-to-physical translation.

Q148. Segmentation divides a process's memory into:

- A) Fixed-size frames
- ✓ B) Logically meaningful, variable-sized units like code, stack, and data
- C) A single contiguous block always
- D) Randomly-sized blocks with no meaning

Answer: B — Segments are logical, variable-size units matching program structure.

Q149. Compared to paging, segmentation:

- A) Always eliminates external fragmentation completely
- ✓ B) Can still suffer from external fragmentation due to variable segment sizes
- C) Uses fixed-size blocks like paging
- D) Has no logical meaning at all

Answer: B — Variable-size segments can leave gaps, unlike uniform pages.

Q150. A segment table stores, for each segment:

- A) Only the segment's name
- ✓ B) The segment's base address and limit (size)
- C) The entire process code
- D) Nothing; segment tables don't exist

Answer: B — Segment tables track base + limit for each logical segment.

Q151. Which memory management technique best matches how programmers conceptually think about a program's structure?

- A) Paging
- ✓ B) Segmentation
- C) Hashing
- D) Round Robin

Answer: B — Segmentation directly mirrors logical program units like functions and stacks.

Q152. Fixed partitioning of memory can lead to:

- A) No fragmentation at all
- ✓ B) Internal fragmentation if a process is smaller than its assigned partition
- C) Only external fragmentation, never internal
- D) Guaranteed optimal memory usage

Answer: B — Fixed-size partitions waste space when a process doesn't fill its partition.

Q153. Dynamic partitioning creates a partition:

- A) Of a fixed size determined at OS installation
- ✓ B) Exactly the size a process needs at load time
- C) That never changes size
- D) Only for the OS kernel

Answer: B — Dynamic partitioning tailors partition size to the specific process.

Q154. Which best explains why paging is preferred over pure contiguous allocation in modern systems?

- A) Contiguous allocation has no fragmentation issues
- ✓ B) Paging avoids external fragmentation and allows non-contiguous physical placement of a process's pages
- C) Paging requires less hardware support
- D) Contiguous allocation is always faster

Answer: B — Paging's flexible frame placement is a major advantage over contiguous allocation.

Q155. A resource allocation graph with multiple instances of a resource type and a cycle means:

- A) Deadlock is guaranteed
- ✓ B) Deadlock is possible but not guaranteed — the cycle is necessary but not sufficient
- C) No cycle can ever occur
- D) The resource type doesn't matter

Answer: B — With multiple instances, a cycle alone doesn't guarantee deadlock.

Q156. Which deadlock handling approach requires processes to declare their maximum resource needs in advance?

- A) Deadlock detection
- ✓ B) Deadlock avoidance (Banker's Algorithm)
- C) Deadlock ignorance (Ostrich algorithm)
- D) None of these

Answer: B — The Banker's Algorithm specifically requires declared maximum needs upfront.

Q157. The 'Ostrich algorithm' approach to deadlocks means:

- A) Aggressively preventing all deadlocks
- ✓ B) Ignoring the problem entirely, assuming deadlocks are rare enough to not matter
- C) Detecting and recovering immediately every time
- D) Avoiding deadlocks via the Banker's Algorithm

Answer: B — Some systems (like many general-purpose OSes) simply ignore rare deadlocks (the 'ostrich' approach).

Q158. Which best distinguishes internal from external fragmentation in one line?

- ✓ A) Internal = wasted space inside an allocated block; External = wasted space scattered between blocks
- B) They are the same phenomenon
- C) Internal only happens with segmentation; External only with paging
- D) Neither occurs in modern systems

Answer: A — This is the precise, standard distinction between the two fragmentation types.

Q159. Segmentation with paging (a hybrid approach used in some real systems) aims to:

- A) Eliminate segments entirely
- ✓ B) Combine segmentation's logical structure with paging's fixed-size allocation benefits
- C) Remove the need for a page table
- D) Only apply to real-time systems

Answer: B — Combining both techniques captures logical structure while avoiding external fragmentation.

Q160. A process requesting resources it will never actually use, just to be safe, primarily contributes to:

- A) Efficient resource utilization
- ✓ B) Poor resource utilization under conservative deadlock prevention schemes
- C) Faster CPU scheduling
- D) Better cache performance

Answer: B — Over-requesting resources upfront (as in some prevention schemes) can waste resources.

Q161. Virtual memory allows:

- A) A process to run only if fully loaded into physical RAM
- ✓ B) A process to execute even if not entirely loaded into physical memory
- C) Physical memory to be removed from a system entirely
- D) Only one process to exist at a time

Answer: B — Virtual memory lets processes run larger than or partially outside physical RAM.

Q162. A key benefit of virtual memory is:

- A) It requires programs to fit entirely in RAM
- ✓ B) It allows more processes to fit in memory simultaneously, improving utilization
- C) It eliminates the need for a CPU
- D) It disables multitasking

Answer: B — Virtual memory improves multiprogramming by not requiring full process residency.

Q163. Demand paging means:

- A) All pages are loaded into memory upfront
- ✓ B) A page is loaded into memory only when it is actually referenced
- C) Pages are never loaded into memory
- D) Only the first page of a process is ever used

Answer: B — Demand paging loads pages lazily, on actual reference.

Q164. A page fault occurs when:

- A) A process references a page currently in physical memory
- ✓ B) A process references a page that is not currently in physical memory
- C) A process terminates normally
- D) The CPU has no processes to run

Answer: B — Page faults happen when a referenced page isn't resident in memory.

Q165. After a page fault, the OS must:

- A) Immediately terminate the process
- ✓ B) Find a free frame (or evict one), load the page from disk, and update the page table
- C) Ignore the fault and continue
- D) Delete the page table entirely

Answer: B — Page fault handling involves frame allocation, disk read, and page table update.

Q166. FIFO page replacement evicts:

- A) The most recently used page
- ✓ B) The page that has been in memory the longest, regardless of recent use
- C) A randomly chosen page
- D) The largest page in memory

Answer: B — FIFO evicts based purely on arrival order, not usage recency.

Q167. LRU (Least Recently Used) page replacement evicts:

- A) The page used most recently
- ✓ B) The page that hasn't been accessed for the longest time
- C) A random page
- D) The first page ever loaded, regardless of recent access

Answer: B — LRU evicts based on how long ago a page was last accessed.

Q168. The Optimal page replacement algorithm evicts:

- ✓ A) The page that will not be used for the longest time in the future
- B) The oldest page in memory
- C) A random page
- D) The page most recently used

Answer: A — Optimal (Belady's) algorithm uses future knowledge to minimize faults — a theoretical benchmark.

Q169. Why can't the Optimal page replacement algorithm be implemented in practice?

- A) It requires too much memory
- ✓ B) It requires knowing future page references in advance, which is generally impossible
- C) It's illegal to implement
- D) It only works with FIFO

Answer: B — Optimal needs future knowledge, making it a theoretical benchmark only.

Q170. Belady's Anomaly refers to the surprising fact that:

- A) More page frames always reduce page faults
- ✓ B) For some algorithms (like FIFO), increasing the number of frames can INCREASE page faults
- C) LRU always suffers from this anomaly
- D) Page faults never occur with more memory

Answer: B — Belady's Anomaly: more frames can paradoxically cause more faults under FIFO.

Q171. Thrashing is best described as:

- A) The CPU running at 100% useful utilization
- ✓ B) A state where the CPU spends more time swapping pages than executing processes
- C) A type of deadlock
- D) A network communication failure

Answer: B — Thrashing = excessive paging activity dominates over useful execution.

Q172. Thrashing typically occurs when:

- A) Too few processes are running relative to available memory
- ✓ B) Too many processes are running relative to available memory, so each lacks enough frames
- C) The disk is completely empty
- D) All pages are permanently resident

Answer: B — Over-multiprogramming without enough memory per process causes thrashing.

Q173. A fix for thrashing is to:

- A) Increase the degree of multiprogramming further
- ✓ B) Reduce the degree of multiprogramming or use the working set model
- C) Disable paging entirely
- D) Ignore it; thrashing resolves itself

Answer: B — Reducing concurrent processes or better frame allocation (working set) addresses thrashing.

Q174. The Working Set of a process is defined as:

- A) Every page the process will ever use
- ✓ B) The set of pages referenced in the most recent time window (delta)
- C) Only the process's code segment
- D) The total size of physical memory

Answer: B — Working set = recently referenced pages within a defined time window.

Q175. The Working Set Model allocates frames to a process based on:

- A) A fixed number regardless of actual need
- ✓ B) Its actual recent memory usage pattern (its working set size)
- C) Random allocation
- D) The process's priority number only

Answer: B — Frame allocation is tied to the process's actual working set size.

Q176. A file system's main responsibility is to:

- A) Manage CPU scheduling
- ✓ B) Organize, name, store, and retrieve data on a storage device
- C) Handle network packet routing
- D) Compile source code

Answer: B — File systems manage data organization and retrieval on storage.

Q177. Contiguous file allocation stores a file's blocks:

- A) Scattered randomly across the disk
- ✓ B) In a single run of consecutive blocks
- C) Only in RAM, never on disk
- D) As a single unallocated block

Answer: B — Contiguous allocation uses one unbroken sequence of blocks per file.

Q178. A drawback of contiguous file allocation is:

- A) Slow sequential access
- ✓ B) External fragmentation and difficulty growing a file
- C) It's incompatible with all file systems
- D) It requires no disk space at all

Answer: B — Fixed contiguous runs fragment free space and complicate file growth.

Q179. Linked file allocation stores each block with:

- ✓ A) A pointer to the next block of the file
- B) No connection to other blocks
- C) A copy of the entire file
- D) A fixed physical location that never changes

Answer: A — Linked allocation chains blocks together via pointers.

Q180. A drawback of linked file allocation is:

- A) Fast random access
- ✓ B) Slow random access, since you must follow pointers from the start
- C) No support for growing files
- D) Guaranteed contiguous storage

Answer: B — Following a pointer chain from the start makes random access slow.

Q181. Indexed file allocation uses:

- ✓ A) A dedicated index block storing pointers to all the file's data blocks
- B) No structure at all
- C) Only contiguous blocks
- D) A single pointer per file with no index

Answer: A — An index block centralizes pointers, enabling fast direct access.

Q182. The inode structure in Unix/Linux file systems is based on the concept of:

- A) Contiguous allocation only
- ✓ B) Indexed allocation
- C) Linked allocation only
- D) No allocation method at all

Answer: B — Inodes are the classic real-world implementation of indexed allocation.

Q183. A single-level directory structure:

- A) Scales well to thousands of files across many users
- ✓ B) Stores all files for all users in one flat directory, causing naming collisions at scale
- C) Supports unlimited nested subdirectories
- D) Is the structure used by all modern OSes

Answer: B — Single-level directories are simple but unworkable beyond a handful of files.

Q184. A tree-structured directory allows:

- A) Only a single flat list of files
- ✓ B) Directories to contain subdirectories to any depth for hierarchical organization
- C) Exactly two levels of directories only
- D) No file organization at all

Answer: B — Tree structures let directories nest to any depth — the modern standard.

Q185. An acyclic-graph directory structure allows:

- A) No sharing of files between directories
- ✓ B) Shared subdirectories/files to appear in multiple places via links
- C) Only single-level directories
- D) Files to exist without any directory

Answer: B — Acyclic-graph directories support shared links across multiple locations.

Q186. Disk scheduling is needed primarily because:

- ✓ A) Disk seek time is the slowest, most significant part of a disk operation
- B) CPUs are too slow to issue requests quickly
- C) Memory access is slower than disk access
- D) Networks require scheduling instead

Answer: A — Seek time dominates disk I/O cost, motivating smart scheduling.

Q187. FCFS disk scheduling services requests:

- A) In order of shortest seek time first
- ✓ B) Strictly in arrival order
- C) In a sweeping elevator pattern
- D) Randomly

Answer: B — FCFS disk scheduling simply follows arrival order, like FCFS CPU scheduling.

Q188. SSTF (Shortest Seek Time First) disk scheduling selects:

- A) The request farthest from the current head position
- ✓ B) The pending request closest to the current head position
- C) Requests in arrival order only
- D) A random request

Answer: B — SSTF always jumps to the nearest pending request.

Q189. A drawback of SSTF disk scheduling is:

- A) It always causes the least total head movement, with no downside
- ✓ B) It can starve requests far from the current 'hot zone' of activity
- C) It never causes starvation
- D) It ignores seek time entirely

Answer: B — Distant requests can be repeatedly skipped, causing starvation risk.

Q190. SCAN ('elevator') disk scheduling works by:

- A) Randomly choosing which request to service next
- ✓ B) Sweeping the head in one direction to the end, then reversing, servicing requests along the way
- C) Only servicing the closest request each time
- D) Ignoring head position entirely

Answer: B — SCAN mimics an elevator, sweeping across the disk and back.

Q191. C-SCAN (Circular SCAN) differs from SCAN in that:

- A) It never reverses direction
- ✓ B) After reaching one end, it jumps back to the start without servicing requests on the return trip, then sweeps forward again
- C) It services requests in both directions on every pass
- D) It is identical to SSTF

Answer: B — C-SCAN always scans in one direction, providing more uniform wait times.

Q192. LOOK disk scheduling differs from SCAN in that:

- A) It always goes all the way to the physical end of the disk
- ✓ B) It only travels as far as the last actual pending request in that direction
- C) It ignores all pending requests
- D) It is identical to FCFS

Answer: B — LOOK stops at the last real request instead of the physical disk boundary.

Q193. Which disk scheduling algorithm generally provides the most uniform (fair) wait times?

- A) FCFS
- B) SSTF
- ✓ C) C-SCAN
- D) None of these provide fairness

Answer: C — C-SCAN's one-directional, circular sweep gives more uniform wait times than SCAN or SSTF.

Q194. If the disk head is at cylinder 50 and pending requests are 10, 60, 90, 20, SSTF would service next:

- A) 90
- B) 10
- ✓ C) 60
- D) 20

Answer: C — 60 is closest to 50 (distance 10), the smallest among the options.

Q195. A file system's directory structure is best compared to:

- A) A single unsorted pile of documents
- ✓ B) A folder/subfolder hierarchy on a computer desktop
- C) A single continuous stream of bytes with no organization
- D) A network routing table

Answer: B — Tree-structured directories are exactly like nested folders on a desktop.

Q196. Why is indexed allocation considered a good balance between contiguous and linked allocation?

- A) It requires no extra structures at all
- ✓ B) It supports fast direct access without requiring contiguous storage, unlike linked allocation's slow traversal
- C) It always wastes more space than contiguous allocation
- D) It cannot support large files

Answer: B — Indexed allocation gets direct access benefits without contiguous storage requirements.

Q197. The main trade-off in choosing SSTF over SCAN for disk scheduling is:

- A) SSTF is always fairer than SCAN
- ✓ B) SSTF minimizes short-term seek time but risks starving distant requests, unlike the fairer SCAN
- C) SCAN causes more starvation than SSTF
- D) There is no trade-off between them

Answer: B — SSTF optimizes locally but can starve far-away requests; SCAN avoids that at some seek-time cost.

Q198. Which best explains the purpose of the working set model in preventing thrashing?

- A) It guarantees infinite memory for every process
- ✓ B) It only allows a process to run if enough frames exist for its current actively-used pages
- C) It disables paging entirely
- D) It has no relation to thrashing

Answer: B — The working set model ties frame allocation to actual current memory needs, avoiding thrashing.

Q199. A page replacement algorithm that removes the page unlikely to be needed soon, based on recent access history, is:

- A) FIFO
- ✓ B) LRU
- C) Random replacement
- D) FCFS

Answer: B — LRU approximates future behavior using the recent past.

Q200. Which best describes why demand paging improves system performance overall?

- A) It loads every page immediately, using more memory upfront
- ✓ B) It avoids wasting time and memory loading pages that may never actually be used
- C) It disables the page table
- D) It requires no disk at all

Answer: B — Demand paging avoids unnecessary I/O and memory use for unused pages.